ICPC Notebook - HSGhello123

Table of Contents

- Graph Algorithms
  - Dinic (Max Flow)
  - MCMF (Min Cost Max Flow)
- Dynamic Programming Optimization
  - CHT (Convex Hull Trick)
  - LineContainer (Dynamic CHT / Li Chao)
- String
  - Z-Function
  - KMP
- Math
  - Calc Phi
  - Calc all Phis
  - Catalan
  - Expected Value
- Data Structure
  - Chairman Tree
  - LinkCut Tree
  - Fenwick
  - RURQ Fenwick
  - Treap
- Geometry
  - Distance to segment
  - Line equation
  - Pick Theorem
  - Convex Hull

Graph Algorithms

Dinic (Max Flow)

```cpp
struct Dinic {
    struct Edge {
        int u, v, flow, cap;
    };
    int num_node, source, sink;
    vector<int> cid;
    vector<int> dist;
    vector<Edge> edge;
    vector<vector<int>> G;

    void init(int _N, int _S, int _T) {
        num_node = _N;
        source = _S;
        sink = _T;
        cid.resize(num_node + 1);
        dist.resize(num_node + 1);
        G.resize(num_node + 1);
    }

    void addEdge(int u, int v, int c) {
        G[u].push_back((int)edge.size());
        edge.push_back({u, v, 0, c});
        G[v].push_back((int)edge.size());
        edge.push_back({v, u, 0, 0});
    }

    bool BFS() {
        for (int i = 0; i <= num_node; ++i)
            dist[i] = INF;
        dist[source] = 0;
```

```
        queue<int> q;
        q.push(source);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int id : G[u]) {
                int v = edge[id].v;
                int rem = edge[id].cap - edge[id].flow;
                if (!rem || dist[v] != INF) continue;
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
        return dist[sink] != INF;
    }

    int DFS(int u, int cur_delta) {
        if (!cur_delta) return 0;
        if (u == sink) return cur_delta;
        for (; cid[u] < (int)G[u].size(); ++cid[u]) {
            int id = G[u][cid[u]];
            int v = edge[id].v;
            if (dist[v] != dist[u] + 1) continue;
            int delta = DFS(v, min(cur_delta, edge[id].cap - edge[id].flow));
            if (!delta) continue;
            edge[id].flow += delta;
            edge[id ^ 1].flow -= delta;
            return delta;
        }
        return 0;
    }

    int max_flow() {
        int flow = 0;
        while (BFS()) {
            for (int i = 0; i <= num_node; ++i)
                cid[i] = 0;
            int x;
            while ((x = DFS(source, INF)))
                flow += x;
        }
        return flow;
    }
} dinic;
```

MCMF (Min Cost Max Flow)

```
struct MCMF {
    struct Edge {
        int u, v, flow, cap, cost;
    };
    int num_node, source, sink;
    vector<int> dist;
    vector<int> trace;
    vector<bool> inQ;
    vector<Edge> edge;
    vector<vector<int>> G;

    void init(int _N, int _S, int _T) {
        num_node = _N;
        source = _S;
        sink = _T;
        G.resize(num_node + 1);
        dist.resize(num_node + 1);
        trace.resize(num_node + 1);
        inQ.resize(num_node + 1, false);
    }

    void addEdge(int u, int v, int c, int cost) {
        G[u].push_back((int)edge.size());
        edge.push_back({u, v, 0, c, cost});
        G[v].push_back((int)edge.size());
        edge.push_back({v, u, 0, 0, -cost});
    }

    bool SPFA() {
        for (int i = 0; i <= num_node; ++i)
            dist[i] = INF;
```

```
        dist[source] = 0;
        queue<int> q;
        q.push(source);
        inQ[source] = true;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            inQ[u] = false;
            for (int id : G[u]) {
                int v = edge[id].v;
                int rem = edge[id].cap - edge[id].flow;
                if (!rem) continue;
                int cost = edge[id].cost;
                if (dist[v] > dist[u] + cost) {
                    dist[v] = dist[u] + cost;
                    trace[v] = id;
                    if (!inQ[v]) {
                        inQ[v] = true;
                        q.push(v);
                    }
                }
            }
        }
        return dist[sink] != INF;
    }

    int MinCostMaxFlow() {
        int flow = 0, cost = 0;
        while (SPFA()) {
            int v = sink, delta = INF;
            while (v != source) {
                int id = trace[v];
                delta = min(delta, edge[id].cap - edge[id].flow);
                v = edge[id].u;
            }
            flow += delta;
            cost += dist[sink] * delta;
            v = sink;
            while (v != source) {
                int id = trace[v];
                edge[id].flow += delta;
                edge[id ^ 1].flow -= delta;
                v = edge[id].u;
            }
        }
        return cost;
    }
} mcmf;
```

Dynamic Programming Optimization

CHT (Convex Hull Trick)

```
// Querying minimum value (bao tren)
struct line {
    int a, b;
    line(int _a, int _b) : a(_a), b(_b) {}
};

// he so goc giam dan
struct CHT {
    vector<line> stk;

    void addLine(line d) {
        while ((int)stk.size() >= 2) {
            line l1 = stk[(int)stk.size() - 2];
            line l2 = stk[(int)stk.size() - 1];
            line l3 = d;
            if (l3.a == l2.a) {
                if (l3.b <= l2.b) {
                    stk.pop_back();
                    continue;
                } else {
                    break;
```

```
                }
            }
            int b23 = l3.b - l2.b;
            int a23 = l2.a - l3.a;
            int b12 = l2.b - l1.b;
            int a12 = l1.a - l2.a;
            if (a23 < 0) {
                a23 *= -1;
                b23 *= -1;
            }
            if (a12 < 0) {
                a12 *= -1;
                b12 *= -1;
            }
            if (b23 * a12 <= b12 * a23)
                stk.pop_back();
            else
                break;
        }
        if (!stk.empty() && stk.back().a == d.a) {
            if (stk.back().b < d.b)
                return;
            else
                stk.pop_back();
        }
        stk.push_back(d);
    }

    int query(int x) {
        int l = 0, r = (int)stk.size() - 1, pos;
        while (l <= r) {
            int mid = (l + r) >> 1;
            if (!mid) {
                pos = mid;
                l = mid + 1;
                continue;
            }
            line l1 = stk[mid - 1];
            line l2 = stk[mid];
            int b12 = l2.b - l1.b;
            int a12 = l1.a - l2.a;
            if (a12 < 0) {
                a12 *= -1;
                b12 *= -1;
            }
            if (b12 <= x * a12) {
                pos = mid;
                l = mid + 1;
            } else {
                r = mid - 1;
            }
        }
        return stk[pos].a * x + stk[pos].b;
    }
} cht;
```

LineContainer (Dynamic CHT)

```
struct line {
    int a, b;
    mutable int p;
    bool operator < (const line &other) const {
        if (other.a == INF && other.b == INF) return p < other.p;
        return a < other.a;
    }
};

struct LineContainer {
    multiset<line> myLC;

    int divi(int x, int y) {
        return x / y - ((x ^ y) < 0 && x % y);
    }

    bool intersect(multiset<line>::iterator x, multiset<line>::iterator y) {
        if (y == myLC.end()) {
            x->p = INF;
            return false;
```

```
        }
        if (x->a == y->a)
            x->p = (x->b > y->b ? INF : -INF);
        else
            x->p = divi(y->b - x->b, x->a - y->a);
        return x->p >= y->p;
    }

    void add(int a, int b) {
        multiset<line>::iterator x = myLC.insert({a, b, 0}), y = next(x);
        while (intersect(x, y))
            y = myLC.erase(y);
        if (x != myLC.begin()) {
            y = prev(x);
            if (intersect(y, x))
                intersect(y, myLC.erase(x));
        }
        while (y != myLC.begin()) {
            x = prev(y);
            if (intersect(x, y)) {
                intersect(x, myLC.erase(y));
                y = x;
            } else {
                break;
            }
        }
    }

    int query(int x) {
        line ans = *myLC.lower_bound({INF, INF, x});
        return ans.a * x + ans.b;
    }
} LC;
```

String

Z-Function

```
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

KMP

```
vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
            j = pi[j - 1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}
```

```
void KMP(const string &s, const string &t) {
    vector<int> pi = prefix_function(s);
    int ans = 0;
    int j = 0;
    for (int i = 0; i < (int)t.length(); i++) {
        while (j > 0 && t[i] != s[j])
            j = pi[j - 1];
        if (t[i] == s[j])
            j++;
        if (j == (int)s.length())
            ans++;
    }
}
```

Math

Calc Phi

```
int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}
```

Calc all Phis

```
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++)
        phi[i] = i;
    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```

Catalan

```
catalan[n] = 1 / (n + 1) * C(2*n, n);
```

Expected Value

```
E(x) = sum(x * P(x))
```

Data Structure

Chairman Tree

```
class Chairman_Tree {
private:
    struct node{
        int l, r;
        ll val;
        node(): l(0), r(0), val(0){}
    };
    int n, cnt;
```

```
    vector<node> tree;

    int build(int l, int r, vector<int>& cc){
        int id = ++cnt;
        if(l == r){
            tree[id].val = cc[l];
            return id;
        }
        int mid = (l + r) >> 1;
        tree[id].l = build(l, mid, cc);
        tree[id].r = build(mid + 1, r, cc);
        tree[id].val = tree[tree[id].l].val + tree[tree[id].r].val;
        return id;
    }

    int Point_Update(int cur, int l, int r, int pos, int val){
        int id = ++cnt;
        tree[id] = tree[cur];
        if(l == r){
            tree[id].val += val;
            return id;
        }
        int mid = (l + r) >> 1;
        if(pos <= mid){
            tree[id].l = Point_Update(tree[cur].l, l, mid, pos, val);
        }
        else{
            tree[id].r = Point_Update(tree[cur].r, mid + 1, r, pos, val);
        }
        tree[id].val = tree[tree[id].l].val + tree[tree[id].r].val;
        return id;
    }

    ll get(int id, int l, int r, int lo, int hi){
        if(l >= lo && r <= hi) return tree[id].val;
        int mid = (l + r) >> 1;
        ll ans = 0;
        if(lo <= mid) ans += get(tree[id].l, l, mid, lo, hi);
        if(hi > mid) ans += get(tree[id].r, mid + 1, r, lo, hi);
        return ans;
    }

public:
    int init(vector<int>& cc){
        n = cc.size() - 1;
        cnt = 0;
        tree.assign(20 * n, node()); // 4 * n + q * log2(n)
        return build(1, n, cc);
    }

    int Point_Update(int id, int pos, int val){
        return Point_Update(id, 1, n, pos, val);
    }

    ll get(int id, int l, int r){
        return get(id, 1, n, l, r);
    }
}; // 1-based indexed

class Lazy_Chairman_Tree {
private:
    struct node{
        int l, r;
        ll val, laz;
        node(int a = 0, int b = 0, ll c = 0, ll d = 0): l(a), r(b), val(c), laz(d){}
    };
    int n, cnt;
    vector<node> tree;

    int build(int l, int r, vector<int>& a){
        int id = ++cnt;
        if(l == r){
            tree[id].val = a[l];
            return id;
        }
        int mid = (l + r) >> 1;
        tree[id].l = build(l, mid, a);
        tree[id].r = build(mid + 1, r, a);
        tree[id].val = tree[tree[id].l].val + tree[tree[id].r].val;
```

```
        return id;
    }

    int Range_Update(int cur, int l, int r, int lo, int hi, int val){
        int id = ++cnt;
        tree[id] = tree[cur];
        if(l >= lo && r <= hi){
            tree[id].val += (ll)val * (r - l + 1);
            tree[id].laz += val;
            return id;
        }
        int mid = (l + r) >> 1;
        if(lo <= mid){
            tree[id].l = Range_Update(tree[cur].l, l, mid, lo, hi, val);
        }
        if(hi > mid){
            tree[id].r = Range_Update(tree[cur].r, mid + 1, r, lo, hi, val);
        }
        tree[id].val = tree[tree[id].l].val + tree[tree[id].r].val + tree[id].laz * (r - l + 1);
        return id;
    }

    ll get(int cur, int l, int r, int lo, int hi, ll sum){
        if(l >= lo && r <= hi) return tree[cur].val + sum * (r - l + 1);
        int mid = (l + r) >> 1;
        ll ans = 0;
        sum += tree[cur].laz;
        if(lo <= mid) ans += get(tree[cur].l, l, mid, lo, hi, sum);
        if(hi > mid) ans += get(tree[cur].r, mid + 1, r, lo, hi, sum);
        return ans;
    }

public:
    int init(vector<int>& a){
        n = a.size() - 1;
        cnt = 0;
        tree.assign(25 * n, node()); // 4 * n + q * log2(n)
        return build(1, n, a);
    }

    int Range_Update(int ver, int lo, int hi, int val){
        return Range_Update(ver, 1, n, lo, hi, val);
    }

    ll get(int ver, int lo, int hi){
        return get(ver, 1, n, lo, hi, 0);
    }
}; // 1-based indexed
```

LinkCut Tree

```
/* ==========================
 * LinkCut Tree with path add + path sum
 * Node indices: 1..N
 * How to use:
 * - Create: LinkCut_Tree L;
 * - Init number of nodes: L.init(N);
 * - Set value of node i to v: L.set_val(i, v);
 * - Get value of node i: L.get_val(i);
 * - Check if u and v are connected: L.connected(u, v);
 * - Link node u and v if they weren't connected L.link(u, v);
 * - Cut node u and v if they are connected: L.cut(u, v);
 * - Add delta to all node on the path from u to v: L.path_add(u, v, delta);
 * - Get sum of all node on the path from u to v: L.path_sum(u, v);
 * ========================== */

class LinkCut_Tree {
private:
    struct Node {
        int ch[2];
        int p;
        bool rev;
        ll val, sum, add;
        int sz;
        Node() : p(0), rev(false), val(0), sum(0), add(0), sz(1) {
            ch[0] = ch[1] = 0;
        }
    };
```

```
    int n;
    vector<Node> tr;

    bool is_root(int x) {
        int p = tr[x].p;
        if(!p) return 1;
        return tr[p].ch[0] != x && tr[p].ch[1] != x;
    }

    void apply_add(int x, ll d) {
        if(!x) return;
        tr[x].add += d;
        tr[x].val += d;
        tr[x].sum += d * tr[x].sz;
    }

    void apply_rev(int x) {
        if(!x) return;
        tr[x].rev ^= 1;
        swap(tr[x].ch[0], tr[x].ch[1]);
    }

    void push(int x) {
        if(!x) return;
        if(tr[x].add != 0) {
            if(tr[x].ch[0]) apply_add(tr[x].ch[0], tr[x].add);
            if(tr[x].ch[1]) apply_add(tr[x].ch[1], tr[x].add);
            tr[x].add = 0;
        }
        if(tr[x].rev) {
            if(tr[x].ch[0]) apply_rev(tr[x].ch[0]);
            if(tr[x].ch[1]) apply_rev(tr[x].ch[1]);
            tr[x].rev = false;
        }
    }

    void pull(int x) {
        if (!x) return;
        tr[x].sz = 1;
        tr[x].sum = tr[x].val;
        if(tr[x].ch[0]) {
            tr[x].sz += tr[tr[x].ch[0]].sz;
            tr[x].sum += tr[tr[x].ch[0]].sum;
        }
        if(tr[x].ch[1]) {
            tr[x].sz += tr[tr[x].ch[1]].sz;
            tr[x].sum += tr[tr[x].ch[1]].sum;
        }
    }

    void rotate(int x) {
        int p = tr[x].p;
        int g = tr[p].p;
        int dir = (tr[p].ch[1] == x);
        int b = tr[x].ch[dir ^ 1];
        if(!is_root(p)) {
            if (tr[g].ch[0] == p) tr[g].ch[0] = x;
            else tr[g].ch[1] = x;
        }
        tr[x].p = g;
        tr[x].ch[dir ^ 1] = p;
        tr[p].p = x;
        tr[p].ch[dir] = b;
        if(b) tr[b].p = p;
        pull(p);
        pull(x);
    }

    void push_path(int x) {
        if(!is_root(x)) push_path(tr[x].p);
        push(x);
    }

    void splay(int x) {
        push_path(x);
        while(!is_root(x)) {
            int p = tr[x].p;
            int g = tr[p].p;
            if(!is_root(p)) {
```

```
                if((tr[p].ch[0] == x) ^ (tr[g].ch[0] == p)) rotate(x);
                else rotate(p);
            }
            rotate(x);
        }
    }

    int access(int x) {
        int last = 0;
        for(int y = x; y; y = tr[y].p) {
            splay(y);
            tr[y].ch[1] = last;
            pull(y);
            last = y;
        }
        splay(x);
        return last;
    }

public:
    LinkCut_Tree() : n(0) {}

    void init(int _n) {
        n = _n;
        tr.assign(n + 1, Node());
        for(int i = 1; i <= n; ++i) {
            tr[i].ch[0] = tr[i].ch[1] = tr[i].p = 0;
            tr[i].rev = false;
            tr[i].val = tr[i].sum = 0;
            tr[i].add = 0;
            tr[i].sz = 1;
        }
    }

    void set_val(int x, ll v) {
        access(x);
        tr[x].val = v;
        pull(x);
    }

    ll get_val(int x) {
        access(x);
        return tr[x].val;
    }

    void make_root(int x) {
        access(x);
        apply_rev(x);
    }

    int find_root(int x) {
        access(x);
        while(1) {
            push(x);
            if (tr[x].ch[0]) x = tr[x].ch[0];
            else break;
        }
        splay(x);
        return x;
    }

    bool connected(int a, int b) {
        if (a == b) return true;
        return find_root(a) == find_root(b);
    }

    void link(int a, int b) {
        make_root(a);
        if (find_root(b) != a) tr[a].p = b;
    }

    void cut(int a, int b) {
        make_root(a);
        access(b);
        if (tr[b].ch[0] == a && tr[a].ch[1] == 0){
            tr[b].ch[0] = 0;
            tr[a].p = 0;
            pull(b);
        }
```

```
    }

    void path_add(int a, int b, ll delta) {
        make_root(a);
        access(b);
        apply_add(b, delta);
    }

    ll path_sum(int a, int b) {
        make_root(a);
        access(b);
        return tr[b].sum;
    }
};
```

Fenwick

```
/* ==========================
 * Walk on Fenwick Tree
 * - find smallest idx with prefsum >= k (k >= 1) in O(log(n)).
 * - returns n+1 if total < k.
 * ========================== */

struct Fenwick_Walk {
    int n;
    vector<ll> bit;

    void init(int _n) {
        n = _n;
        bit.assign(n + 1, 0);
    }

    void add(int idx, ll val) {
        for (; idx <= n; idx += idx & -idx) bit[idx] += val;
    }

    ll sum(int idx) {
        ll r = 0;
        for (; idx > 0; idx -= idx & -idx) r += bit[idx];
        return r;
    }

    ll range_sum(int l, int r) {
        if (l > r) return 0;
        return sum(r) - sum(l - 1);
    }

    int find_kth(ll k) {
        if (k <= 0) return 1;
        int idx = 0;
        int LOG = 1;
        while((1 << LOG) <= n) ++LOG;
        for(int d = LOG; d >= 0; --d) {
            int nxt = idx + (1 << d);
            if(nxt <= n && bit[nxt] < k) {
                idx = nxt;
                k -= bit[nxt];
            }
        }
        return idx + 1;
    }
};
```

RURQ Fenwick

```
class RURQ_Fenwick_Tree {
private:
    int n;
    vector<ll> Tree1, Tree2;

    inline ll Get_Sum(vector<ll> &Tree, int pos) {
        ll ans = 0;
        for (; pos > 0; pos -= pos & (-pos)) {
            ans += Tree[pos];
        }
        return ans;
```

```
    }

public:
    void init(int len) {
        n = len;
        Tree1.assign(n + 1, 0);
        Tree2.assign(n + 1, 0);
    }

    void Point_Update(vector<ll> &Tree, int pos, ll num) {
        for (; pos <= n; pos += pos & (-pos)) {
            Tree[pos] += num;
        }
    }

    void Range_Update(int l, int r, ll num) {
        Point_Update(Tree1, l, (n - l + 1) * num);
        Point_Update(Tree1, r + 1, -(n - r) * num);
        Point_Update(Tree2, l, num);
        Point_Update(Tree2, r + 1, -num);
    }

    ll Pref_Sum(int pos) {
        return Get_Sum(Tree1, pos) - Get_Sum(Tree2, pos) * (n - pos);
    }

    ll Range_Sum_Query(int l, int r) {
        return Pref_Sum(r) - Pref_Sum(l - 1);
    }
};
```

Treap

```
/* ==========================
 * FHQ Treap.
 * Public API positions are 1-indexed.
 * How to use:
 * - init with 1-based vector: init_from_vector(const vector& a)
 * - init with 0-based vector: init_from_vector_0(const vector& v)
 * - insert value before pos (1-indexed). If pos = n+1, append: insert_at(int pos, ll value)
 * - erase value at pos (1-indexed): erase_at(int pos)
 * - reverse range [l,r]: range_reverse(int lpos, int rpos)
 * - range add: add delta to all elements in [l,r]: range_add(int lpos, int rpos, ll delta)
 * - range set: assign value v to all elements in [l,r]: range_set(int lpos, int rpos, ll v)
 * - range sum query: range_sum(int lpos, int rpos)
 * - get element at pos: at(int pos)
 * - set single element at pos to v: set_at(int pos, ll v)
 * ========================== */

class FHQ_Treap {
private:
    struct node {
        int l = 0, r = 0;
        int pr = 0;
        int sz = 1;
        ll val = 0, sum = 0;
        bool rev = false;
        ll lazy_add = 0;
        bool has_set = false;
        ll lazy_set = 0;
        node() {}
        node(ll v, int p)
            : l(0), r(0), pr(p), sz(1), val(v), sum(v), rev(false), lazy_add(0), has_set(false), lazy_set(0)
{}
    };
    vector<node> tr;
    int root = 0;
    mt19937 rng;

    int new_node(ll v) {
        int pr = uniform_int_distribution<int>(1, INT_MAX)(rng);
        tr.push_back(node(v, pr));
        return tr.size() - 1;
    }

    void pull(int id) {
        if(!id) return;
        tr[id].sz = 1;
```

```
        tr[id].sum = tr[id].val;
        if(tr[id].l) {
            tr[id].sz += tr[tr[id].l].sz;
            tr[id].sum += tr[tr[id].l].sum;
        }
        if(tr[id].r) {
            tr[id].sz += tr[tr[id].r].sz;
            tr[id].sum += tr[tr[id].r].sum;
        }
    }

    void apply_set(int id, ll v) {
        if(!id) return;
        tr[id].has_set = 1;
        tr[id].lazy_set = v;
        tr[id].lazy_add = 0;
        tr[id].val = v;
        tr[id].sum = v * tr[id].sz;
    }

    void apply_add(int id, ll d) {
        if(!id) return;
        if(tr[id].has_set) {
            tr[id].lazy_set += d;
            tr[id].val += d;
        }
        else {
            tr[id].lazy_add += d;
            tr[id].val += d;
        }
        tr[id].sum += d * tr[id].sz;
    }

    void apply_rev(int id) {
        if(!id) return;
        tr[id].rev ^= 1;
        swap(tr[id].l, tr[id].r);
    }

    void push(int id) {
        if(!id) return;
        if(tr[id].has_set) {
            if(tr[id].l) apply_set(tr[id].l, tr[id].lazy_set);
            if(tr[id].r) apply_set(tr[id].r, tr[id].lazy_set);
            tr[id].has_set = false;
            tr[id].lazy_set = 0;
        }
        if(tr[id].lazy_add != 0) {
            if(tr[id].l) apply_add(tr[id].l, tr[id].lazy_add);
            if(tr[id].r) apply_add(tr[id].r, tr[id].lazy_add);
            tr[id].lazy_add = 0;
        }
        if(tr[id].rev) {
            if(tr[id].l) apply_rev(tr[id].l);
            if(tr[id].r) apply_rev(tr[id].r);
            tr[id].rev = false;
        }
    }

    pair<int, int> split(int id, int k) {
        if(!id) return {0, 0};
        push(id);
        int lsz = tr[id].l ? tr[tr[id].l].sz : 0;
        if(k <= lsz) {
            auto pr = split(tr[id].l, k);
            tr[id].l = pr.second;
            pull(id);
            return {pr.first, id};
        }
        else {
            auto pr = split(tr[id].r, k - lsz - 1);
            tr[id].r = pr.first;
            pull(id);
            return {id, pr.second};
        }
    }

    int merge(int a, int b) {
        if(!a) return b;
```

```
        if(!b) return a;
        if(tr[a].pr > tr[b].pr) {
            push(a);
            tr[a].r = merge(tr[a].r, b);
            pull(a);
            return a;
        }
        else {
            push(b);
            tr[b].l = merge(a, tr[b].l);
            pull(b);
            return b;
        }
    }

public:
    FHQ_Treap() {
        tr.reserve(1 << 20);
        tr.push_back(node());
        rng.seed(chrono::high_resolution_clock::now().time_since_epoch().count());
    }

    void clear() {
        tr.assign(1, node());
        root = 0;
    }

    int size() {
        return root ? tr[root].sz : 0;
    }

    void init_from_vector(const vector<ll> &a) {
        clear();
        int m = (int)a.size();
        if(m <= 1) return;
        vector<int> nodes;
        for(int i = 1; i < m; ++i) {
            int id = new_node(a[i]);
            nodes.push_back(id);
        }
        root = 0;
        for(int id : nodes) root = merge(root, id);
    }

    void init_from_vector_0(const vector<ll> &v) {
        clear();
        int m = (int)v.size();
        if(m == 0) return;
        vector<int> nodes;
        for(int i = 0; i < m; ++i) {
            int id = new_node(v[i]);
            nodes.push_back(id);
        }
        root = 0;
        for(int id : nodes) root = merge(root, id);
    }

    void insert_at(int pos, ll value) {
        int n = size();
        if(pos < 1) pos = 1;
        if(pos > n + 1) pos = n + 1;
        auto p = split(root, pos - 1);
        int nn = new_node(value);
        root = merge(merge(p.first, nn), p.second);
    }

    void erase_at(int pos) {
        int n = size();
        if(pos < 1 || pos > n) return;
        auto a = split(root, pos - 1);
        auto b = split(a.second, 1);
        root = merge(a.first, b.second);
    }

    void range_reverse(int lpos, int rpos) {
        if(lpos > rpos) return;
        auto a = split(root, lpos - 1);
        auto b = split(a.second, rpos - lpos + 1);
        apply_rev(b.first);
```

```
        root = merge(a.first, merge(b.first, b.second));
    }

    void range_add(int lpos, int rpos, ll delta) {
        if(lpos > rpos) return;
        auto a = split(root, lpos - 1);
        auto b = split(a.second, rpos - lpos + 1);
        apply_add(b.first, delta);
        root = merge(a.first, merge(b.first, b.second));
    }

    void range_set(int lpos, int rpos, ll v) {
        if(lpos > rpos) return;
        auto a = split(root, lpos - 1);
        auto b = split(a.second, rpos - lpos + 1);
        apply_set(b.first, v);
        root = merge(a.first, merge(b.first, b.second));
    }

    ll range_sum(int lpos, int rpos) {
        if(lpos > rpos) return 0;
        auto a = split(root, lpos - 1);
        auto b = split(a.second, rpos - lpos + 1);
        ll ans = b.first ? tr[b.first].sum : 0;
        root = merge(a.first, merge(b.first, b.second));
        return ans;
    }

    ll at(int pos) {
        if(pos < 1 || pos > size()) return 0;
        auto a = split(root, pos - 1);
        auto b = split(a.second, 1);
        ll ans = b.first ? tr[b.first].val : 0;
        root = merge(a.first, merge(b.first, b.second));
        return ans;
    }

    void set_at(int pos, ll v) {
        if(pos < 1 || pos > size()) return;
        auto a = split(root, pos - 1);
        auto b = split(a.second, 1);
        if(b.first) {
            tr[b.first].val = v;
            tr[b.first].sum = v;
        }
        root = merge(a.first, merge(b.first, b.second));
    }
};
```

Geometry

Distance to segment

```
#define x first
#define y second
typedef pair<int, int> pii;

int dot(pii A, pii B, pii C) {
    pii AB, AC;
    AB.x = B.x - A.x;
    AB.y = B.y - A.y;
    AC.x = C.x - A.x;
    AC.y = C.y - A.y;
    return AB.x * AC.x + AB.y * AC.y;
}

// Compute the cross product AB x AC
int cross(pii A, pii B, pii C) {
    pii AB, AC;
    AB.x = B.x - A.x;
    AB.y = B.y - A.y;
    AC.x = C.x - A.x;
    AC.y = C.y - A.y;
    return AB.x * AC.y - AB.y * AC.x;
```

```
}

// Compute the distance from A to B
double distance(pii A, pii B) {
    int dx = A.x - B.x;
    int dy = A.y - B.y;
    return sqrt(dx * dx + dy * dy);
}

// Compute the distance from AB to C
// if isSegment is true, AB is a segment, not a line.
double linePointDist(pii A, pii B, pii C, bool isSegment) {
    double dist = abs(cross(A, B, C)) / distance(A, B);
    if (isSegment) {
        int dot1 = dot(B, A, C);
        if (dot1 < 0) return distance(B, C);
        int dot2 = dot(A, B, C);
        if (dot2 < 0) return distance(A, C);
    }
    return dist;
}
```

Line Equation

```
// Line equation of given two points (x1, y1), (x2, y2)
// A = y2 - y1;
// B = x1 - x2;
// C = A * x1 + B * y1

// Intersection of two lines
void intersect() {
    double det = A1 * B2 - A2 * B1;
    if (det == 0) {
        // Lines are parallel or coincident
        if (A1 * C2 == A2 * C1) {
            // Lines are coincident
        }
        else {
            // Lines are parallel
        }
    }
    else {
        double x = (B2 * C1 - B1 * C2) / det;
        double y = (A1 * C2 - A2 * C1) / det;
    }
}

// Perpendicular line
// Ax + By = C
// -Bx + Ay = D
```

Pick Theorem

```
// S=I+B/2-1
// Inside: I
// On edge: B

int N;
ll area, boundary, inside, x[maxN], y[maxN];

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%lld %lld", &x[i], &y[i]);
    x[N] = x[0]; y[N] = y[0];
    for(int i = 0; i < N; i++){
        area += x[i] * y[i+1];
        area -= y[i] * x[i+1];
    }
    area = abs(area);
    for(int i = 0; i < N; i++){
        if(x[i+1] == x[i]) boundary += abs(y[i+1]-y[i]);
        else if(y[i+1] == y[i]) boundary += abs(x[i+1]-x[i]);
        else boundary += __gcd(abs(x[i+1]-x[i]), abs(y[i+1]-y[i]));
    }
    printf("%lld %lld\n", (area+2-boundary)/2, boundary);
```

```
}
```

Convex Hull

```
#include <bits/stdc++.h>
using namespace std;

struct Point {
    int x, y;
};

// A -> B -> C counterclockwise
bool ccw(const Point &A, const Point &B, const Point &C) {
    return 1LL * (B.x - A.x) * (C.y - A.y) - 1LL * (C.x - A.x) * (B.y - A.y) > 0;
}

vector<Point> convexHull(vector<Point> p, int n) {
    sort(p.begin(), p.end(), [](const Point &A, const Point &B) {
        if (A.x != B.x) return A.x < B.x;
        return A.y < B.y;
    });

    vector<Point> hull;
    hull.push_back(p[0]);

    for (int i = 1; i < n; ++i) {
        while (hull.size() >= 2 && ccw(hull[hull.size() - 2], hull.back(), p[i])) {
            hull.pop_back();
        }
        hull.push_back(p[i]);
    }

    for (int i = n - 2; i >= 0; --i) {
        while (hull.size() >= 2 && ccw(hull[hull.size() - 2], hull.back(), p[i])) {
            hull.pop_back();
        }
        hull.push_back(p[i]);
    }

    if (n > 1) hull.pop_back();
    return hull;
}

int main() {
    int n;
    cin >> n;
    vector<Point> p(n);
    for (Point &a : p) {
        cin >> a.x >> a.y;
    }
    vector<Point> hull = convexHull(p, n);
    cout << hull.size() << '\n';
    for (Point p : hull) {
        cout << p.x << ' ' << p.y << '\n';
    }
}
```